

SCHOOL MANAGEMENT SYSTEM

BY :KARRAR HAYDER ABDULHUSSIN

Instructor:Mahdi molaei

Software lab

An Annotated Product Backlog and Complete User Story List

A Windows desktop application written in Python

Part 1 presents the initial product backlog of the School Management System we plan to build, story by story, with a comment after each story pointing out what works, what does not, and what needs discussion. **Part 2** is the complete, cleaned-up list of user stories with acceptance criteria and priorities, ready to be used as a working backlog.

The structure of Part 1 follows the style of Mike Cohn's annotated product backlogs: a story, then a candid comment. The system is a School Management System that will be built as a Windows desktop application written in Python, and its requirements are expressed as user stories.

Part 1: The Annotated Backlog

About This Backlog

The School Management System brings the educational, financial and administrative work of a school into one program. A School Manager sets up the academic year, grade levels, approved courses, classes, tuition and schedules. Teachers record attendance and grades. Students attend classes. Parents follow their child's progress, pay tuition and may ask for a tuition correction. An Admin looks after accounts, access and system logs.

Because this is a Windows desktop application, a few decisions need attention early. Mobile access is out of scope. A "session" depends on whether workstations share a database. And paying online means a desktop program must hand the user to a bank page and get the result back. These issues appear in the critiques below because stories exist to start the conversations that settle them.

A Danger in Critiquing Stories

Critiquing a story from outside the team is easy and slightly unfair. Stories are written to start conversations, and the critic was not in the room. A story that looks vague may be perfectly clear to a team that has already agreed what "manage" or "report" means.

These stories are written before any conversations take place, which is why the weak ones are weak. In a real project, a story's wording is only the first half; the second half is what the product owner and the developers say to each other. So read the critiques as suggestions for what to talk about, not as verdicts.

Don't Strive for Perfect Stories

Nobody writes perfect stories, and trying wastes time. A good-enough story starts a useful conversation; the details can then move into acceptance criteria. The aim of this document is to show a range: some stories are fine; some need splitting and a few should not be stories at all. The initial draft deliberately includes typical mistakes so that the comments can show how to fix them.

How to Read the Stories

Each backlog item is shown in a blue box. If a rewrite is worthwhile, it appears below the critique in a green box, with acceptance criteria where they help.

A few conventions are used throughout:

- Stories prefer "I can ..." to "I want ...". An "I can" story is done only when the user truly can do the thing.
- Job stories ("When ... I want ... so that ...") are used where the situation matters more than the person.
- Details such as field lists, error messages and numeric limits belong in acceptance criteria.
- Statements about the whole system (speed, security, platform) are non-functional requirements and are not written as stories.

The five roles are:

Role	Responsibility
Admin	Technical owner of the system. Manages users and roles, monitors access, defines new managers, views system logs and corrects financial transactions affected by errors.
School Manager	Runs the school in the system: academic years, grade levels, courses, classes, teachers, enrollment, schedules, tuition, reports and tuition requests.
Teacher	Sees assigned classes and student lists, records and corrects attendance and grades.
Student	Joins an active class, with attendance recorded automatically.
Parent	Views the child's report, pays tuition, views transaction history and asks for tuition corrections.

Accounts and Access

Every role in the system begins here. These stories look simple, which is exactly why several of them hide decisions the team must make before coding.

> As a user, I want to log in with my username and password so that I can use the system.

The role of "user" is acceptable here because every one of the five roles really does sign in the same way. The weak part is the so-that clause: "use the system" tells the team nothing. The real benefit is that each person sees only what their role allows.

The remaining details (the dashboard shown for the role, an error for bad credentials, an access-history entry) are the conversation and the acceptance criteria, not the story.

> **Better:** As a registered user, I can sign in so that I reach the features that belong to my role.

Acceptance criteria:

- Valid credentials open the dashboard for the user's role
- Invalid credentials show one generic error message and keep the user on the login screen
- Every attempt and its final outcome is written to the access history

> As a user, I want my account to be blocked after 3 wrong password attempts for 15 minutes, or until the administrator unblocks it, so that nobody can guess my password.

This is a real need, but "3" and "15 minutes" are numbers that will be debated and changed. A rule like this is easily repeated in several places (login, security, exception handling), which is a warning sign: duplicated requirements drift apart. A single story is enough.

It is also a better fit for a job story, because the trigger (repeated failures) matters more than the person.

> **Better:** When I enter wrong credentials several times in a row, my account is temporarily locked so that attackers cannot guess my password.

Acceptance criteria:

- The account is blocked after 3 consecutive failed attempts
- It becomes active again after 15 minutes or when the Admin approves
- The user cannot sign in while blocked

> As a user, I want a password of 6 to 10 characters containing a capital letter, a digit and a special character so that my account is secure.

This one should not be a story at all; it is a non-functional requirement. It also teaches a lesson about copying rules without questioning them: capping a password at 10 characters makes accounts easier to attack, not safer. A minimum length with no tiny maximum is the modern approach.

Notice how easily a rule like this slips into a backlog because it "sounds like security". Challenge it in the conversation instead.

> As a user, I want to sign in through the central SSO service so that I do not need another password.

Here the story names a solution (SSO) rather than a need. If nobody could sign in whenever the SSO service is down, a Windows desktop application used inside a school would carry a heavy dependency. A better approach is to make central sign-in an optional, lower-priority story and keep local accounts as the default so the product works on day one.

Administration

The admin is the technical owner of the system, not the person who runs the school. Keeping those two roles apart is a deliberate design decision.

> As an admin, I can create a school manager account and assign it to a specific school so that the school can start using the system.

This is a decent story. The phrase "specific school" does raise a question worth discussing early: is one installation used by one school, or by many? A Windows desktop application is naturally single-school unless it connects to a shared database. If the answer is one school, the word "assign" may disappear entirely.

The fields to collect (name, national ID, contact details) belong in the acceptance criteria.

> As an admin, I can manage users and roles so that access is correct.

"Manage" is a slippery word. Does it mean create, edit, delete, change roles, reset passwords? Until the details are settled the story cannot be estimated.

When a story contains manage, maintain or administer, split it or list the actions as acceptance criteria.

> **Better:** As an admin, I can manage user accounts and their roles so that every user has access only to the parts that match their role.

Acceptance criteria:

- Each user has exactly one role
- The Admin panel lists the users and their roles

> As an admin, I can see the list of all active sessions so that I can monitor access.

Before this story is ready the team must ask what a "session" is in a desktop application. If each workstation runs its own copy against its own local database, there is nothing to list. If several workstations share a database server, sessions are a real thing. The story is only valid under the second architecture, so the architecture decision comes first.

> As an admin, I can edit any financial transaction so that I can fix mistakes.

This is the riskiest story in the backlog, and it reads far too casually. "Any" and "fix mistakes" give the Admin power to rewrite money records. Even if the intent is legitimate (a system error produced a wrong amount), the story needs acceptance criteria about when it may be used.

The story should also require a reason for the edit. The reason is the heart of this story, so it belongs in it rather than being left to chance.

> **Better:** As an admin, I can correct a tuition transaction affected by a system error so that parents' balances are right.

Acceptance criteria:

- The Admin finds the transaction by searching
- The new amount and a reason for the edit must be entered
- The reason is recorded with the correction

Academic Structure

These stories describe the school's setup each year: academic year, grade levels, approved courses and classes. Almost all are performed by the School Manager and most only a few times a year.

> As a school manager, I can define a new academic year with a start and end date so that the school can begin a new cycle.

Good story. The rule that dates must not overlap an earlier year is a business rule that belongs in the acceptance criteria, not in a second story. Notice also that choosing which year is "current" is a separate piece of behavior. It will be written as its own small story.

> As a school manager, I can define grade levels (such as grade 1 to grade 6) for each academic year so that classes can be organized.

The example in parentheses is helpful and harmless. Because levels are defined per academic year, the story also implies that the order of the levels matters; that is an acceptance criterion, not a separate story.

> As a school manager, I can register the approved courses issued by the Ministry of Education for each grade level so that teachers and schedules use official subjects.

The so-that clause is good because it explains why the word "approved" matters. Because rules about approved courses may change during the year, the system should let course lists be changed without editing code. That is a non-functional requirement attached to this story, not a new story.

> As a school manager, I can create physical or virtual classes for a grade level, with a name and capacity, so that students can be placed in them.

"Physical or virtual" quietly adds a requirement: do virtual classes behave differently? If they do (for example a meeting link or an online room), that deserves its own conversation, because it affects class participation. If they do not, the words can be dropped.

Capacity is stored for each class, but nothing says what happens when a class is full. That is a question for the conversation, and the answer belongs in the acceptance criteria.

> As a school manager, I can set a numeric tuition amount for every class so that parents see what they owe.

"Numeric" is a UI detail that adds nothing; of course it is a number. A more useful question is the unit: Rials or Tomans, and are decimals possible? Also, this story is really part of the tuition capability, and placing it with the other money stories would make the backlog easier to read.

Teachers and Scheduling

Scheduling is the most algorithmic part of the system and the place where a Python implementation will spend the most effort.

> As a school manager, I can create a teacher account with name, national ID and specialty so that the teacher can use the system.

The list of fields should be acceptance criteria. Another question for the conversation: should a duplicate national ID be rejected for teachers the way it will be for students?

> As a school manager, I can schedule courses so that each class has a weekly program.

This is an epic wearing a story costume. It contains choosing a slot, choosing a teacher, detecting conflicts, and showing the result. Written this way it cannot be estimated.

Splitting it gives four stories: assign a course and teacher to a slot; prevent conflicts; show the weekly program of a class; show the weekly program of a teacher.

> As a school manager, I want to see a warning if a teacher or class is double-booked so that I avoid conflicts.

The story says "warning", but a warning you can ignore and a block you cannot are very different behaviors. Left undecided, this ambiguity could easily survive until testing.

Decide in the conversation, then state it as a job story: when a slot would double-book a teacher or a class, the system stops me and shows which slot conflicts.

> **Better:** When I place a course in a slot that conflicts with another, the system stops me and shows the conflicting entry so that no teacher or class is double-booked.

Acceptance criteria:

- Conflict is checked for both the teacher and the class
- The message names the day, hour and class or teacher involved

> As a school manager, I can assign a teacher to a class and a course so that the teacher knows what to teach.

This overlaps heavily with scheduling, because both place a teacher in a class. Overlapping stories cause people to build the same feature twice or to argue about which one is done. Merge them, or make the difference explicit: assigning a teacher to teach a course in a class, versus placing it at a time.

Student Enrollment

Enrollment connects people, classes and money, so a single weak story here causes problems in later sections.

> As a school manager, I can register a new student with personal information and assign them to a class so that they are enrolled.

Two actions are glued together: creating the student and placing them in a class. "Personal information" is also undefined. A good conversation here would produce a list of required fields, which are then acceptance criteria.

Better to split it into: register a student; place a student in a class; the system creates the student's account automatically.

> As a school manager, I want an error message when I enter an existing national ID so that duplicates are prevented.

This is not a story; it is an acceptance criterion for registering a student. The user does not wish for error messages. They wish to register students without creating duplicates. Writing every validation as its own story would inflate the backlog by dozens of items.

Class Participation and Attendance

Attendance is recorded in two ways: automatically when a student joins a session, and manually by a teacher.

> As a student, I can pick my active class from the dashboard and enter it so that my attendance is recorded automatically.

This story depends on something nobody wrote down: what is "entering a class" in a desktop application? Does the application host a live room, open a meeting link, or simply show a page for that session? The answer decides whether automatic attendance is easy (the app knows when the student clicked) or impossible to trust (the student joined an external meeting).

A class can only be entered after the teacher has started it, so we also need to decide how a class is started. The student's manual "record attendance" option is a good fallback to keep as an acceptance criterion.

> As a teacher, I can mark each student present or absent, and I can use a "mark all present" button, so that attendance is recorded quickly.

The first half is good. The "mark all present" button is a UI shortcut and does not need to be in the story. It still makes a nice acceptance criterion because it records an intention: most students are present most of the time, so make the common case fast.

It would also help to say what happens when the automatic record from a student and the teacher's manual choice disagree. A good default is that the teacher's decision wins.

> As a teacher, I can correct a previously recorded attendance status so that mistakes are fixed.

Sensible, and it mirrors the grade-correction story. The real questions are who may correct, and how long afterward. These are things to settle in the conversation, not in the story.

Grades

Grades are simple to enter and politically sensitive to change.

> As a teacher, I can enter each student's grade for every course so that results are recorded.

"Every course" is not needed because a teacher enters grades for the courses they teach. The more important gap is the grading scheme: numbers from 0 to 20? Letters? Several exams per term? Defining exam types may turn out to be a hidden story.

Do not try to settle those details now. Write the story, add "grade scale" as a topic for the conversation, and revisit when it nears the top of the backlog.

> As a teacher, I can correct a wrong grade so that the right value is recorded.

Fine on its own, and it needs few words. It pairs naturally with the story about entering grades, since both are performed by the teacher on the same screen. It could even be an acceptance criterion of the entering story, but keeping it separate makes the correction visible in the backlog.

Parent Access and Reports

This area is where privacy matters most, because it involves minors' records.

> As a parent, I enter my child's national ID to see their academic and disciplinary report so that I know how my child is doing.

This describes a mechanism (typing the ID) when the need is simply viewing my child's report. If a parent is already linked to a child, why must they type an ID at all? One answer is a second check to protect against a stolen session; if that is the reason, say so.

A good security rule is to show the same "access denied" message whether the ID doesn't exist or belongs to someone else, so attackers cannot learn which IDs are real. That detail is valuable and belongs in the acceptance criteria.

> **Better:** As a parent, I can view the academic and disciplinary report of my child so that I can follow their progress.

Acceptance criteria:

- The report is shown only for a student linked to my account
- Any invalid or unlinked national ID produces the same "access denied" message

Tuition and Payments

Money stories need more care than most. The Windows desktop context matters here, because card payments normally happen in a browser page that the desktop application has to hand off to and return from.

> As a parent, I can pay tuition so that the student's financial status becomes "paid".

The so-that clause describes a database change rather than a benefit. What the parent wants is a settled bill. Also, nobody has decided how the payment happens: a bank gateway page opened from the app, a local wallet, a cash entry by the manager? Each is a very different amount of work.

Cancel, failure and retry flows should be acceptance criteria or separate small stories, not squeezed into the happy path.

> As a parent, if a payment fails or the gateway does not respond, the transaction must not be recorded as final so that my balance stays correct.

A good job story hiding in a plain story. When the payment result is unknown, the transaction stays Pending or Failed and is never recorded as final. Exception handling for payments should state this clearly, so it is worth promoting to a story of its own.

> As a parent, I want every transaction to have a unique tracking code so that duplicates are avoided.

This is an implementation rule, not a user need. Parents wish not to be charged twice; tracking codes are one way to achieve that. Keep the user's goal in the story and leave the mechanism to acceptance criteria.

> As a parent, I can ask for my tuition amount to be changed by giving a proposed amount and a reason so that mistakes or hardship can be considered.

This is a good story, since the so-that clause names two situations that justify it. The review side exists in a separate manager story, so both halves of the interaction are covered, which is something many backlogs forget.

Reports

Reporting is where weak stories hide because nobody wants to argue about reports until they are needed.

> As a school manager, I can see reports so that I can manage the school.

A classic. Which reports, with which columns? The system needs three (financial, educational, attendance) and three filters (date, class, grade level). Writing three stories, one per report, is much easier than writing one story and guessing. The "manage the school" so-that clause adds nothing and can be dropped.

Exports to PDF and Excel and printing on A4 could be added as acceptance criteria once the screens work.

Non-Functional Requirements

Many statements about the whole system are tempting to write as stories. As stories they feel forced.

> As a developer, I want the average API response time to be under 500 ms so that users are not bored.

A desktop application with a local database does not have an "API" in the sense meant here, and "developer" is not a user. Rewrite this as a non-functional requirement about what users feel: screens open in under two seconds; reports of fewer than 1,000 records appear in under three.

> As a user, I want the system to support 10,000 simultaneous users so that it can handle peak load.

For a single school this number is wildly out of proportion. Ten thousand concurrent users suggests a national platform, not a school application that runs on office PCs. Replace it with a number based on a real school: perhaps a few hundred students and staff, with maybe fifty active at one time.

> As a user, I want the system to work on mobile devices so that I can access it anywhere.

This conflicts with the decision to build a Windows desktop application. Rather than quietly ignoring it, remove it from scope in writing and, if wanted, add "mobile access" as a future idea. Unstated scope changes are the usual reason projects slip.

What This Backlog Teaches

Looking across the initial backlog, the same few mistakes repeat, as Cohn also observes: once one story is written a certain way, its neighbors copy it.

- **Details inside the story.** Field lists, numeric limits and error messages creep into many stories and belong in acceptance criteria.
- **Mechanisms instead of needs.** Typing a national ID, using SSO, and tracking codes are ways of meeting a need, not the need.
- **Slippery words.** Manage, report and schedule each hide several stories.
- **Requirements that are not stories.** Performance, security rules and platform choices work better as plain non-functional statements.
- **Hidden architecture questions.** Sessions, class participation and online payments cannot be estimated until the team decides how a desktop application will do them.

Part 2 applies these lessons to produce a complete backlog.

Part 2: Complete User Story List

This list contains the user stories for the School Management System, the system we plan to build. Stories are grouped by epic. Priorities use MoSCoW: **Must** (needed for the system to be usable), **Should** (important but not blocking), **Could** (nice to have).

Stories marked "(Job story)" use the "When ... I want ... so that ..." form because the trigger is more important than the role. A story's acceptance criteria are a starting point for conversation and may be refined when the story approaches the top of the backlog.

Backlog Summary

Epic	Stories	Must	Should	Could
Accounts and Access (AUTH)	2	1	0	1
System Administration (ADM)	2	0	2	0
Academic Structure (ACA)	2	1	0	0
Teachers (TCH)	1	0	0	0
Scheduling (SCH)	2	0	0	0
Student Enrollment (ENR)	2	1	0	0
Class Participation and Attendance (ATT)	2	1	2	0
Grades (GRD)	1	2	0	0
Parent Access and Student Report (PAR)	1	1	0	0
Tuition and Payments (TUI)	3	2	2	0
Tuition Correction Requests (TCR)	1	1	0	0
Management Reports (REP)	1	2	1	0
Total	**20**	**12**	**7**	**1**

Accounts and Access (AUTH)

ID	User Story	Acceptance Criteria	Priority
AUTH-01	As a registered user, I can sign in with my username and password so that I reach the features that belong to my role.	• Valid credentials open the dashboard of the user's role • Invalid credentials show an error message and return the user to the login step • The access history records the user's information and the final login status	Must
AUTH-02	When I enter wrong credentials three times, my account is blocked so that nobody can guess my password. (Job story)	• The account is blocked after 3 failed attempts • It becomes active again after 15 minutes or with the Admin's approval • The user cannot sign in while blocked	Must

Grades (GRD)

ID	User Story	Acceptance Criteria	Priority
GRD-01	As a teacher, I can enter a grade for each student in a course so that results are recorded.	• The teacher selects the course and class • The student list is shown • The system saves the grades of the course	Must
GRD-02	As a teacher, I can correct a wrong grade so that the right value is recorded.	• The grade must already be recorded • The new grade replaces the old one	Must

System Administration (ADM)

ID	User Story	Acceptance Criteria	Priority
ADM-01	As an admin, I can create a School Manager account and assign it to a specific school so that the school can start using the system.	• The Admin enters the manager's information and the school • The system creates an account with the Manager role	Must
ADM-04	As an admin, I can view the list of all active sessions so that I can monitor access.	• The list is shown in the "Access Management" section	Must

Parent Access and Student Report (PAR)

ID	User Story	Acceptance Criteria	Priority
PAR-01	As a parent, I can view the academic and disciplinary report of my child by entering the child's national ID so that I can follow their progress.	• The system checks that the national ID belongs to my child • The comprehensive report is shown	Must
PAR-02	As a parent, when I enter a national ID that does not exist or is not my child's, I see the same "access denied" message so that nobody can discover which IDs exist. (Job story)	• The message is the same in both cases • The system never says whether the national ID exists	Must

Academic Structure (ACA)

ID	User Story	Acceptance Criteria	Priority
ACA-01	As a school manager, I can define a new academic year with a start and end date so that the school can begin a new cycle.	• Start and end dates are required • A year whose dates overlap an earlier year is rejected with an error message	Must
ACA-05	As a school manager, I can create a class (for example class A or B) with a name and capacity for a grade level so that students can be placed in it.	• Each class belongs to one grade level • Grade levels and approved courses must be defined first	Must

Teachers (TCH)

ID	User Story	Acceptance Criteria	Priority
TCH-01	As a school manager, I can create a teacher account with name, national ID and specialty so that the teacher can use the system.	• The system creates an account with the Teacher role	Must

Scheduling (SCH)

ID	User Story	Acceptance Criteria	Priority
SCH-01	As a school manager, I can assign a course and its teacher to a day and hour slot of a class so that the weekly program is defined.	• Classes, teachers and approved courses must exist first • The weekly program is saved	Must

ID	User Story	Acceptance Criteria	Priority
SCH-02	When I place a course in a slot that conflicts with another, the system stops me and shows a warning so that no teacher or class is double-booked. (Job story)	• Conflict is checked for both the teacher and the class • The conflicting schedule is not saved	Must

Student Enrollment (ENR)

ID	User Story	Acceptance Criteria	Priority
ENR-01	As a school manager, I can register a new student with personal information so that the student exists in the system.	• The student's personal information and national ID are entered • A duplicate national ID is rejected with an error message	Must
ENR-02	As a school manager, I can place a student in a class so that the student is enrolled for the academic year.	• The academic year, grade levels, classes and class schedules must already exist • Each student belongs to exactly one class • Each class belongs to one grade level	Must

Class Participation and Attendance (ATT)

ID	User Story	Acceptance Criteria	Priority
ATT-01	As a student, I can pick my active class from my dashboard and enter it so that I can take part in the lesson.	• Only a class that the teacher has started is available • The class screen opens	Must
ATT-05	As a teacher, I can mark each student of a session as present or absent so that attendance is complete.	• The teacher selects the class and course from the weekly program • The session's schedule must be active • The system saves the attendance of all students	Must

Tuition and Payments (TUI)

ID	User Story	Acceptance Criteria	Priority
TUI-01	As a parent, I can see the tuition amount due for my child so that I know what to pay.	• The tuition must have been defined by the School Manager	Must
TUI-02	As a parent, I can confirm payment of my child's tuition so that the bill is settled.	• I select the tuition payment section • The system shows the amount • On confirmation the system records the transaction and changes the student's financial status to "Paid"	Must
TUI-03	When I cancel during payment, the process is cancelled and returns to the first step so that I am not charged. (Job story)	• No amount is deducted	Must
TUI-04	When a payment fails or the payment service does not respond, the transaction is not recorded as final so that my balance stays correct. (Job story)	• The transaction status stays Pending or Failed unless payment is confirmed	Must
TUI-05	When the connection drops during registration or payment, the data is kept and sent again automatically so that nothing is lost. (Job story)	• Data is stored in a temporary queue • After reconnecting the operation completes automatically • Duplicate records are prevented	Should
TUI-06	As a parent, I am protected from duplicate payments so that I am not charged twice.	• Every transaction has a unique tracking code	Must
TUI-07	As a parent, I can view my transaction history so that I can track what I paid.	• The history is available on the Parent dashboard	Should

Tuition Correction Requests (TCR)

ID	User Story	Acceptance Criteria	Priority
TCR-01	As a parent, I can request a change to my child's tuition by giving a proposed amount and a reason so that mistakes can be considered.	• A tuition transaction exists for the current year • The request is saved with the status "Awaiting manager approval"	Must
TCR-02	As a school manager, I am notified of a new correction request so that I can handle it.	• The notification is visible to the manager	Must
TCR-03	As a school manager, I can review the details of a request so that I can decide.	• The manager sees the proposed amount and the reason	Must
TCR-04	As a school manager, I can approve a request so that the tuition amount is corrected.	• The system updates the transaction amount	Must
TCR-05	As a school manager, I can reject a request and give a reason so that the decision is clear.	• A reason is required • The request is closed	Must

Management Reports (REP)

ID	User Story	Acceptance Criteria	Priority
REP-01	As a school manager, I can view a financial report so that I know the state of tuition.	• Available in the "Reports" section	Must
REP-02	As a school manager, I can view an educational report so that I can follow academic performance.	• Available in the "Reports" section	Must
REP-03	As a school manager, I can view an attendance report so that I can follow absences.	• Available in the "Reports" section	Must
REP-04	As a school manager, I can filter reports by date, class or grade level so that I see only what I need.	• Filters can be chosen before the report is shown	Must
REP-05	As a school manager, I can produce reports in PDF or Excel format and print them so that I can share them.	• A4 printing is supported	Should

Non-Functional Requirements

As the first part explained, these are written as plain statements rather than stories. They apply to the whole system and should be checked whenever a related story is accepted.

ID	Area	Requirement
NFR-01	**Platform**	Runs on Windows 10 and Windows 11 as a desktop application written in Python; mobile devices are not part of this version.
NFR-02	**Performance**	Average response under 500 ms; main screen loads in under 2 seconds; reports with fewer than 1,000 records are produced in under 3 seconds; the number of simultaneous users is sized for one school.
NFR-03	**Reliability**	Data is queued during network failures and sent again automatically; every payment and enrollment has a unique tracking code to prevent duplicates; data is stored in a database safe against failure.
NFR-04	**Availability**	Available at all hours with at least 99% uptime; a backup is activated within 2 minutes after a power or server failure; on serious errors (for example no database response) new operations are blocked and a suitable message is shown.
NFR-05	**Security**	All data exchange is encrypted; each user reaches only the sections of their role; accounts are blocked after 3 failed sign-ins; passwords contain an uppercase letter, a digit and a special character, with a minimum length and no unreasonably small maximum (see Part 1).
NFR-06	**Backup**	Automatic backup of all financial transactions and user data every 24 hours, archived for 5 years.
NFR-07	**Maintainability**	Modular, layered architecture; changes in educational rules (courses, tuition) are made from the Admin panel without changing code; complete technical documentation is provided.
NFR-08	**Usability**	Simple, menu-based interface; larger and more readable display for teacher screens; optional interactive help for each feature.
NFR-09	**Printing**	Reports are printed on A4 paper.

https://github.com/Karrar19988/SCHOOL-MANAGEMENT-SYSTEM